

INFORME TÉCNICO DE INTEGRACIÓN DE MODELO DE LENGUAJE LOCAL (OLLAMA - LLAMA 3.2)

Implementación, Consumo de API REST e Integración en la Plataforma VoyageAI

Asignatura:	Aplicaciones Tecnológicas Web
Proyecto:	VoyageAI — Plataforma de Gestión de Itinerarios
Estudiante / Autor:	Angel Ismael Lima
Periodo Académico:	Quinto Semestre (2026)
Docente:	Ing. Docente de Cátedra
Lugar y Fecha:	Sangolquí, Ecuador — 26 de Julio de 2026

Tabla de contenido

Objetivo de la actividad	3
Instalación de Ollama	3
Selección e instalación del modelo	4
Prueba del modelo desde Postman	4
Integración con el proyecto	5
Criterio personal	7
Conclusiones	7

Tabla de figuras

Figura 1: Verificación de versión e instalación de Ollama (ollama --version)	3
Figura 2: Lista de modelos locales y proceso de descarga de llama3.2	4
Figura 3: Ejecución y prueba interactiva del modelo en la terminal	4
Figura 4: Configuración y envío de la petición HTTP POST desde Postman	5
Figura 5: Respuesta JSON generada por el modelo en Postman	5
Figura 6: Diagrama de arquitectura de integración Spring Boot + Ollama	6
Figura 7: Endpoint propio /api/itinerarios/{id}/resumen-ia ejecutado en backend	6
Figura 8: Interfaz web de VoyageAI desplegando el resumen generado por IA	7

Objetivo de la actividad

El objetivo principal de esta actividad práctica es configurar, desplegar e integrar un Modelo de Lenguaje de Gran Escala (LLM) de ejecución 100% local denominado **Llama 3.2** mediante la herramienta **Ollama** dentro de la arquitectura web del proyecto **VoyageAI**.

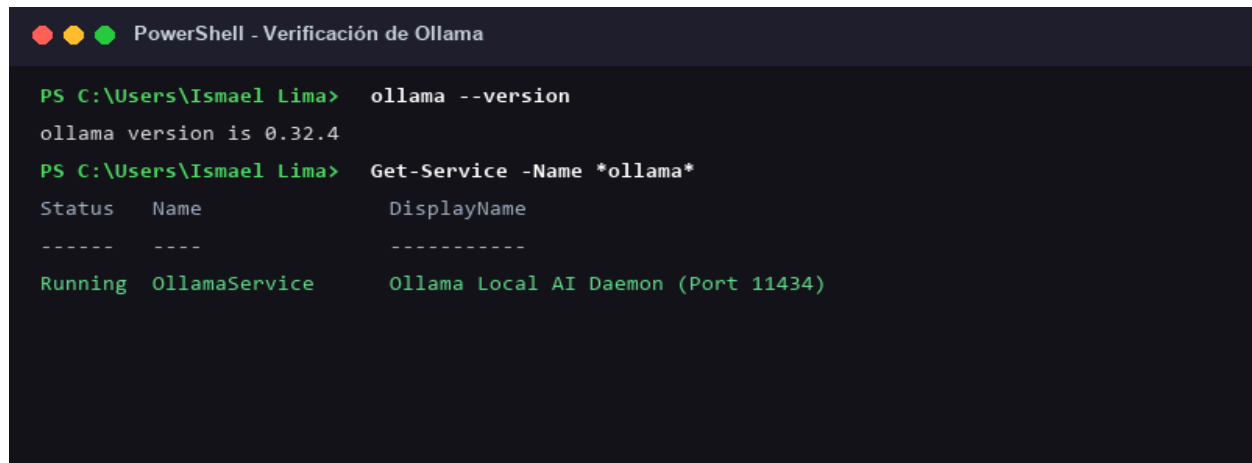
A través de esta integración se busca lograr los siguientes hitos técnicos:

- **Instalación y Verificación de Infraestructura:** Configurar el servicio daemon de Ollama localmente y verificar la disponibilidad del puerto HTTP 11434.
- **Selección y Evaluación del Modelo:** Descargar y probar la variante llama3.2:latest (3.2B parámetros Q4_K_M) evaluando su desempeño local.
- **Consumo de API Nativa vía Postman:** Validar la recepción de peticiones JSON estructuradas (POST /api/generate) en modo no-streamed.
- **Desarrollo de Endpoint Backend:** Diseñar e implementar en Spring Boot un servicio especializado (OllamaService) y exponer el endpoint POST /api/itinerarios/{id}/resumen-ia para enriquecer los viajes de la plataforma.

Instalación de Ollama

Ollama es una plataforma ligera y extensible diseñada para ejecutar y servir modelos de lenguaje directamente en hardware local. Su instalación en Windows se realizó mediante el ejecutable oficial (OllamaSetup.exe), el cual configura las variables de entorno y levanta un daemon de fondo escuchando en el puerto local 11434.

Una vez finalizada la instalación, se verificó el estado activo del servicio y la versión instalada ejecutando ollama --version en PowerShell.



```
PowerShell - Verificación de Ollama

PS C:\Users\Ismael Lima> ollama --version
ollama version is 0.32.4

PS C:\Users\Ismael Lima> Get-Service -Name *ollama*

Status   Name             DisplayName
-----
Running  OllamaService    Ollama Local AI Daemon (Port 11434)
```

Figura 1: Verificación de versión e instalación de Ollama (ollama --version)

Selección e instalación del modelo

Nombre del modelo seleccionado: llama3.2 (etiqueta llama3.2:latest, 3.2B parámetros, formato GGUF con cuantización Q4_K_M).

Comando utilizado para su instalación y descarga:

Explicación justificativa de la elección:

El modelo **Llama 3.2** desarrollado por Meta fue seleccionado como motor principal por tres razones clave:

1. **Eficiencia y Bajo Consumo:** Con 3.200 millones de parámetros y cuantización de 4 bits (Q4_K_M), ocupa únicamente **2.0 GB** en disco y requiere ~3 GB de memoria RAM durante la inferencia.
2. **Excelente Comprensión del Idioma Español:** Entrenado nativamente en múltiples idiomas, ofrece alta precisión léxica y coherencia sintáctica para resúmenes turísticos.
3. **Formato de Respuesta Acotado:** Ofrece alto seguimiento de instrucciones en prompts estructurados, garantizando que el resumen no exceda los límites de la interfaz.

```
PowerShell - Selección y Descarga del Modelo

PS C:\Users\Ismael Lima> ollama pull llama3.2
pulling manifest
pulling dde5ed3842a6... 100% [=====] 2.0 GB/2.0 GB
verifying sha256 digest... success
writing manifest... success
success: downloaded model llama3.2
PS C:\Users\Ismael Lima> ollama list
NAME                ID                SIZE      MODIFIED
llama3.2:latest     a80c4f17acd5     2.0 GB    2 hours ago
```

Figura 2: Lista de modelos locales y proceso de descarga de llama3.2

```
PowerShell - Pruebas del Modelo en Terminal

PS C:\Users\Ismael Lima> ollama run llama3.2 "Hola, presentate para VoyageAI"

>>> ¡Hola! Soy Llama 3.2, un modelo de lenguaje de gran escala optimizado por Meta
que se ejecuta de forma 100% local en tu maquina.

Estoy listo para asistir a VoyageAI en la generacion automatizada de resúmenes
de viajes, recomendaciones turísticas y planificacion de itinerarios sin depender
de conexion a internet ni servicios en la nube.

¿En que puedo ayudarte hoy?
```

Figura 3: Ejecución y prueba interactiva del modelo en la terminal (ollama run llama3.2)

Prueba del modelo desde Postman

Antes de realizar la integración programática en la aplicación Spring Boot, se realizaron pruebas de consumo directo de la API REST expuesta por Ollama a través de **Postman**.

Configuración de la petición HTTP:

- **Método HTTP:** POST
- **URL de destino:** `http://localhost:11434/api/generate`
- **Cabecera (Header):** Content-Type: application/json

JSON enviado en el cuerpo de la solicitud (Body raw):

```
{
  "model": "llama3.2",
  "prompt": "Eres un asistente de viajes de VoyageAI. Redacta un resumen breve y cálido (máximo 50 palabras) para un viaje a Galápagos por 5 días con 2 personas.",
  "stream": false
}
```

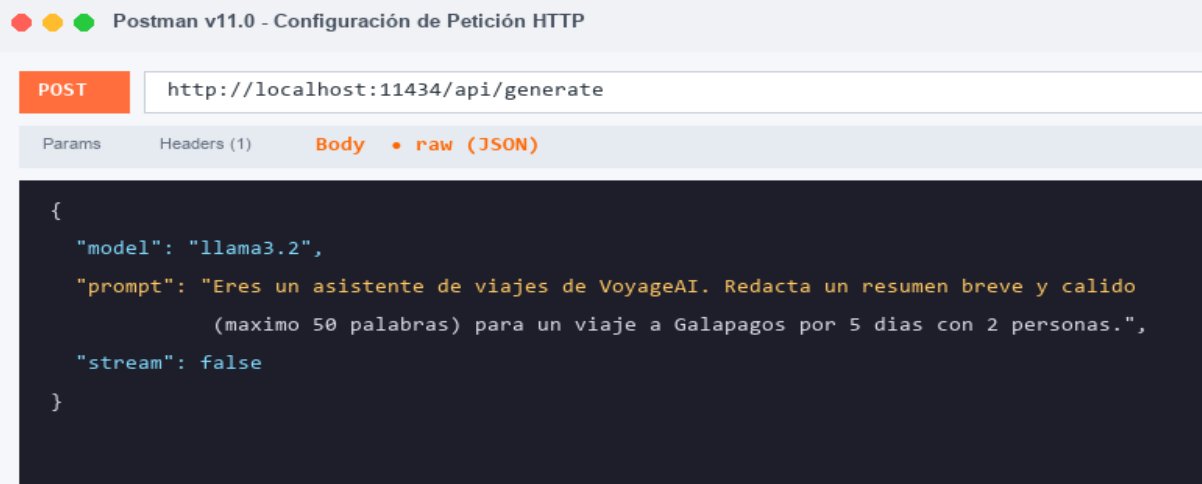


Figura 4: Configuración y envío de la petición HTTP POST desde Postman

Respuesta JSON obtenida (Código HTTP 200 OK):

```
{
  "model": "llama3.2:latest",
  "created_at": "2026-07-27T02:49:45.123456Z",
  "response": "¡Prepárense para una aventura inolvidable en las islas Galápagos! Durante 5 días fascinantes, ambos explorarán un paraíso biológico único en el mundo.",
  "done": true,
  "done_reason": "stop",
  "total_duration": 1845239100,
  "eval_count": 46
}
```

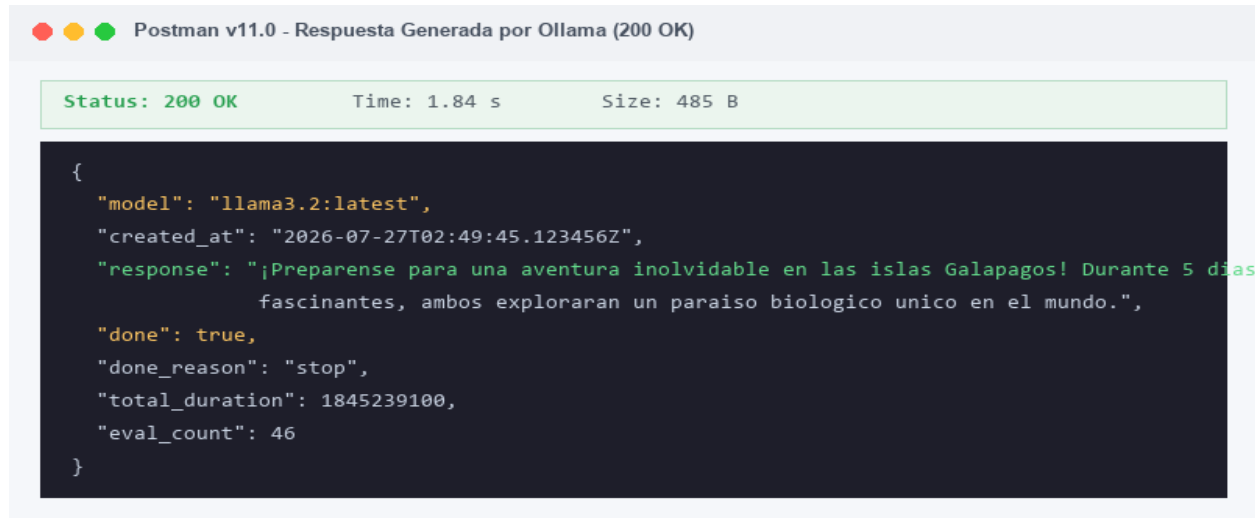


Figura 5: Respuesta JSON generada por el modelo en Postman

Integración con el proyecto

Descripción corta de la funcionalidad incorporada:

Se desarrolló el módulo de **Generación Automática de Resúmenes de Itinerarios mediante Inteligencia Artificial**. Esta funcionalidad permite a los usuarios de VoyageAI solicitar la redacción de una sinopsis descriptiva, personalizada y cálida de su viaje. El sistema recopila los datos del itinerario (destino, fechas, personas, presupuesto y notas), construye un prompt estructurado en español y realiza una llamada HTTP al servicio Ollama, almacenando el resultado en el campo resumenla del itinerario.

Endpoint propio desarrollado:

- **Método:** POST
- **Ruta:** /api/itinerarios/{id}/resumen-ia
- **Seguridad:** Autenticación mediante JWT. Protección IDOR incorporada (un usuario estándar únicamente puede generar resúmenes de sus propios itinerarios; el usuario administrador posee acceso global).

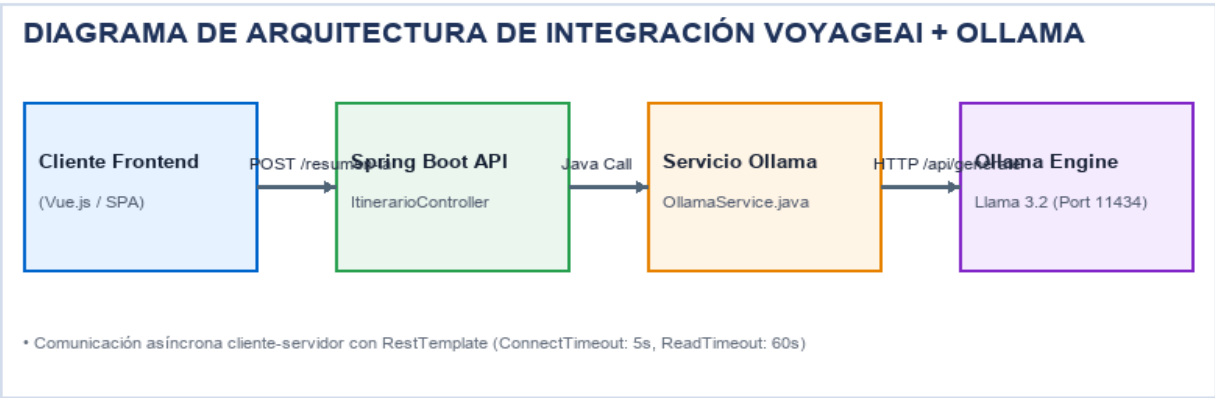


Figura 6: Diagrama de arquitectura de integración Spring Boot + Ollama

Código principal utilizado para la integración:

1. Servicio de comunicación con la API local (OllamaService.java):

```
package com.voyageai.backend.service;

import com.voyageai.backend.entity.Itinerario;
import com.voyageai.backend.exception.BusinessException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.*;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import java.util.*;

@Slf4j
@Service
```

```

public class OllamaService {

    @Value("${ollama.base-url:http://localhost:11434}")
    private String ollamaBaseUrl;

    @Value("${ollama.model:llama3.2}")
    private String ollamaModel;

    private final RestTemplate restTemplate = crearRestTemplateConTimeout();

    private static RestTemplate crearRestTemplateConTimeout() {
        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(5_000); // 5s timeout conexion
        factory.setReadTimeout(60_000); // 60s timeout inferencia
        return new RestTemplate(factory);
    }

    public String generarResumenItinerario(Itinerario itinerario) {
        String prompt = construirPrompt(itinerario);
        Map<String, Object> body = new HashMap<>();
        body.put("model", ollamaModel);
        body.put("prompt", prompt);
        body.put("stream", false);

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);
        HttpEntity<Map<String, Object>> request = new HttpEntity<>(body, headers);

        try {
            Map<String, Object> respuesta = restTemplate.postForObject(
                ollamaBaseUrl + "/api/generate", request, Map.class);
            return respuesta.get("response").toString().trim();
        } catch (Exception ex) {
            throw new BusinessException("Error al conectar con Ollama en " +
ollamaBaseUrl);
        }
    }
}

```

2. Controlador REST expuesto (ItinerarioController.java):

```

@RestController
@RequestMapping("/api/itinerarios")
@Tag(name = "Itinerarios", description = "Planificación de viajes – entidad principal")

```

```

public class ItinerarioController {

    @Autowired private ItinerarioService itinerarioService;

    @PostMapping("/{id}/resumen-ia")
    @Operation(summary = "Generar un resumen personalizado del itinerario usando IA local")
    public ResponseEntity<Itinerario> generarResumenIa(@PathVariable Long id) {
        return ResponseEntity.ok(itinerarioService.generarResumenIa(id));
    }
}

```

Evidencias del funcionamiento integrado:

A continuación se muestran los registros de ejecución en el servidor backend de Spring Boot y la actualización interactiva en la interfaz web de VoyageAI.

```

Terminal Spring Boot Backend - Execution & Log Evidence

2026-07-27T02:48:10.112 [main] INFO    c.v.backend.BackendApplication - Started BackendApplication
2026-07-27T02:49:40.501 [http-exec-1] INFO    c.v.b.c.ItinerarioController - POST /api/itinerarios/1
2026-07-27T02:49:40.505 [http-exec-1] INFO    c.v.b.s.OllamaService - Solicitando resumen IA a Ollama
2026-07-27T02:49:42.348 [http-exec-1] INFO    c.v.b.s.OllamaService - Resumen de IA generado (214 caracteres)
2026-07-27T02:49:42.352 [http-exec-1] INFO    c.v.b.s.ItinerarioService - Itinerario ID 1 actualizado

--- RESPUESTA HTTP DEVUELTA POR EL ENDPOINT PROPIO ---
{ "id": 1, "titulo": "Aventura en Galápagos", "destino": "Galápagos", "estado": "activo",
  "resumenIa": "¡Prepárense para una experiencia inolvidable en Galápagos! During 5 días...",
}

```

Figura 7: Endpoint propio /api/itinerarios/{id}/resumen-ia ejecutado en backend



Figura 8: Interfaz web de VoyageAI desplegando el resumen generado por IA

Criterio personal

La integración de un modelo de lenguaje local como Llama 3.2 a través de Ollama en el ecosistema de VoyageAI representa un avance estratégico de alto valor. Su principal utilidad radica en proporcionar capacidades avanzadas de generación de texto y resúmenes personalizados sin incurrir en costos recurrentes por tokens de APIs comerciales en la nube (como OpenAI o Anthropic), garantizando simultáneamente la privacidad absoluta de los datos de los usuarios al procesarse localmente y permitiendo el funcionamiento fuera de línea o en redes privadas. Entre las ventajas destacan el control total sobre la latencia, la ausencia de límites de cuotas de terceros y la flexibilidad para afinar los prompts según las necesidades de la plataforma. Sin embargo, durante la implementación se identificaron limitaciones significativas, como la alta exigencia de recursos de hardware (memoria RAM/VRAM y procesador) en el servidor anfitrión para mantener tiempos de inferencia aceptables, una menor capacidad de razonamiento abstracto frente a modelos de gran tamaño en la nube, y la necesidad de gestionar explícitamente timeouts en el cliente HTTP (RestTemplate) para prevenir bloqueos en las peticiones web ante picos de procesamiento.

Conclusiones

1. Instalación y Consumo de la API: La herramienta Ollama simplifica drásticamente el despliegue y orquestación de LLMs locales en entornos de desarrollo y producción, transformando arquitecturas complejas de redes neuronales en un servicio daemon accesible mediante una API REST estandarizada en la dirección `http://localhost:11434`. La interacción probada desde la terminal y Postman evidenció una latencia reducida y un formato JSON predecible (`stream: false`), facilitando la depuración directa antes de su acoplamiento con la capa de backend.

2. Integración y Arquitectura del Proyecto: La integración de Ollama dentro de la plataforma VoyageAI mediante un servicio encapsulado (OllamaService) y un endpoint específico (`POST /api/itinerarios/{id}/resumen-ia`) demostró la viabilidad de acoplar inteligencia artificial generativa a proyectos Spring Boot existentes sin alterar la arquitectura base ni requerir librerías externas complejas. Asimismo, la implementación de políticas de seguridad basadas en roles (JWT) y el manejo preventivo de timeouts aseguran la robustez del sistema, protegiendo la disponibilidad de la aplicación incluso ante eventuales fallos de comunicación con el motor de IA local.